



Universidade Federal do Rio Grande do Norte

**Implementação em SystemC de um Processador RISCO
com Pipeline de Dois Estágios**

Relatório Técnico

Alunos: Claudivan Costa de Lima Júnior, Flávio Oliveira Silva Júnior, Leonardo Machado Moreira

Professor: Márcio Eduardo Kreutz

1. Introdução

O estudo de arquitetura e organização de computadores constitui uma das bases fundamentais da formação em Engenharia de Computação. Compreender como um processador executa instruções, gerencia registradores, acessa memórias e coordena o fluxo de dados exige mais do que leitura teórica: é necessário construir e observar o funcionamento de uma implementação concreta.

SystemC é uma biblioteca/conjunto de classes e macros em C++ voltada à modelagem e simulação de sistemas de hardware. Neste projeto, ela é utilizada para descrever módulos síncronos e combinacionais de forma hierárquica, permitindo simular o comportamento do processador sem a necessidade de FPGA ou ASIC.

O projeto apresentado neste relatório implementa o datapath básico de um processador denominado RISCO, uma arquitetura RISC simplificada de 32 bits, com ISA própria de 13 instruções, banco de 16 registradores, ULA com 8 operações, duas memórias separadas e pipeline de dois estágios. O testbench cobre todas as instruções da ISA implementada e alguns casos de borda, além de gerar um arquivo VCD para análise no GTKWave.

2. Objetivos

2.1 Objetivo geral

Implementar e validar, em SystemC, o datapath básico de um processador RISCO com pipeline de dois estágios, cobrindo todas as instruções definidas na ISA do projeto.

2.2 Objetivos específicos

1. Implementar a Unidade Lógico-Aritmética (ULA) com suporte a oito operações e geração de flags de status.
2. Implementar o banco de registradores com 16 registradores de 32 bits, três portos de leitura e um porto de escrita.
3. Implementar memórias genéricas separadas para instruções e dados.
4. Implementar a unidade de controle responsável pela decodificação da instrução e pela geração dos sinais de controle.
5. Integrar os módulos no datapath com pipeline de dois estágios (IF e EX/MEM/WB), incluindo cálculo do PC e lógica de flush.
6. Validar todas as instruções da ISA implementada por meio de testbench.
7. Gerar arquivo de rastreamento VCD para visualização das formas de onda no GTKWave.

3. Fundamentação Teórica

3.1 Datapath e unidade de controle

O datapath é o conjunto de componentes de um processador responsável pelo transporte e pela transformação dos dados. Ele inclui registradores, ULA, multiplexadores, memórias

e conexões internas. A unidade de controle interpreta o opcode de cada instrução e gera os sinais que configuram o comportamento do datapath.

3.2 Banco de registradores e ULA

O banco de registradores é um conjunto de posições de armazenamento de alta velocidade internas ao processador. A ULA realiza operações aritméticas e lógicas sobre os operandos fornecidos e gera sinais de status. Neste projeto, a flag zero é usada pela instrução BEQ, enquanto carry e negativo ficam disponíveis para observação e possíveis extensões futuras.

3.3 Memória, pipeline e hazards

A separação entre memória de instruções e memória de dados segue uma organização do tipo Harvard. Essa separação favorece a execução em pipeline, pois permite modelar acessos independentes a instruções e dados dentro do mesmo ciclo de clock.

Pipeline é uma técnica que divide a execução de instruções em etapas sobrepostas. No processador RISCO, há dois estágios: busca de instrução (IF) e execução completa (EX/MEM/WB). Hazards são situações em que essa sobreposição pode produzir resultados incorretos, especialmente hazards de dados do tipo RAW e hazards de controle causados por branch e jump.

4. Arquitetura Geral do Processador

O processador RISCO é organizado em dois estágios de pipeline.

4.1 Estágio 1 — Busca de instrução (IF)

No estágio IF, o contador de programa (PC) fornece o endereço à memória de instruções. A instrução lida é armazenada no registrador de pipeline IF/EX, que a transfere ao estágio seguinte na próxima borda de clock. O PC pode ser atualizado para PC+1, PC+offset em um BEQ tomado, ou para o imediato estendido em um JUMP absoluto.

4.2 Estágio 2 — Execução completa (EX/MEM/WB)

No estágio EX/MEM/WB, a instrução mantida no registrador IF/EX é decodificada pela unidade de controle. O banco de registradores fornece os operandos, a ULA executa a operação solicitada, a memória de dados é acessada em LOAD e STORE, e o resultado final é selecionado pelo multiplexador de write-back.

4.3 Diagrama do datapath

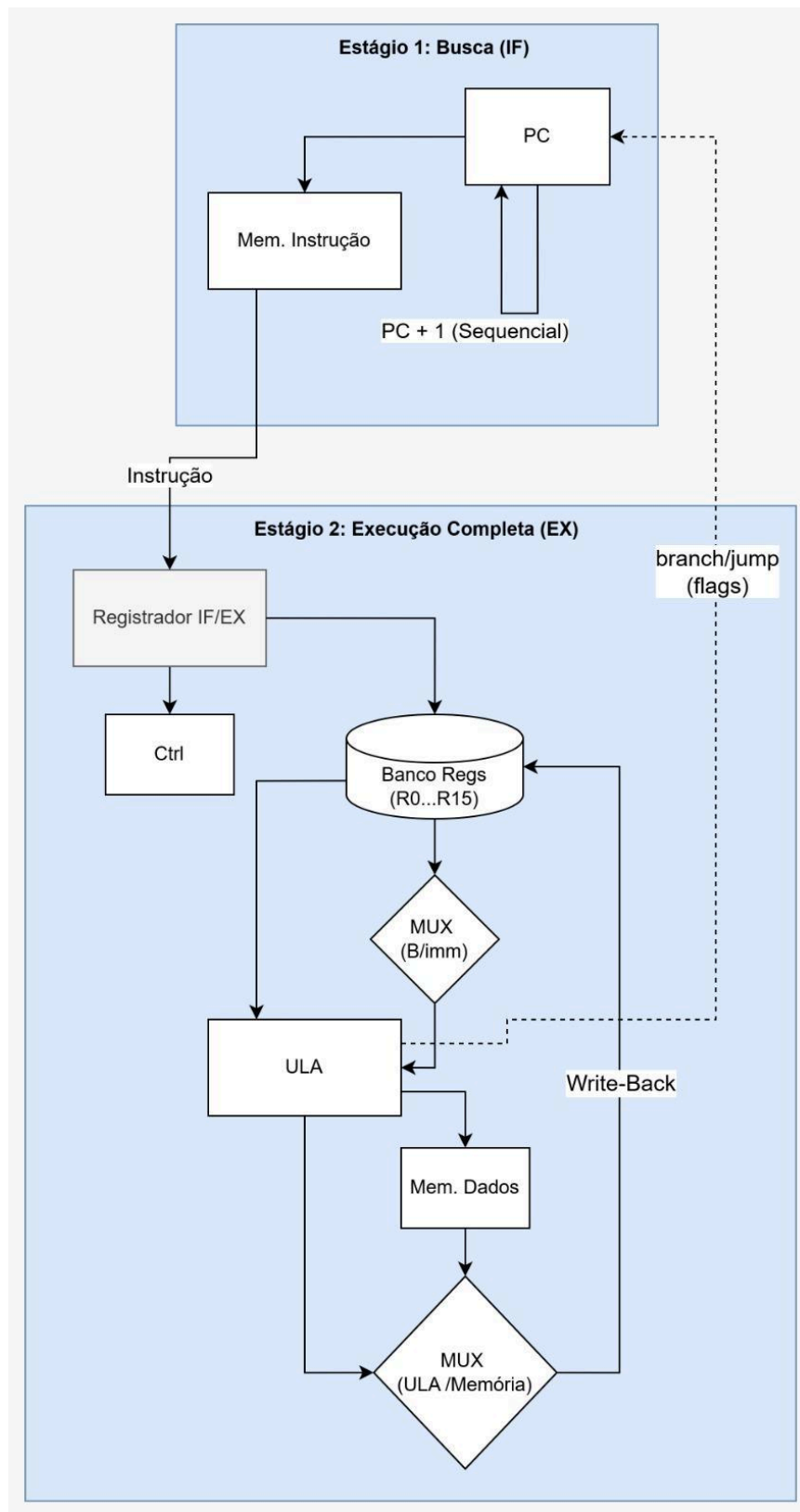


Figura 1 — Datapath do processador RISCO com pipeline de dois estágios.

O diagrama representa a arquitetura geral do projeto, com PC, memória de instruções, registrador IF/EX, unidade de controle, banco de registradores, ULA, memória de dados e

write-back. A implementação final possui duas adições importantes não totalmente representadas no diagrama: o terceiro porto de leitura no banco de registradores e um MUX adicional antes do operando A da ULA para a instrução BEQ.

5. Conjunto de Instruções — ISA

A ISA do processador RISCO define 13 instruções, divididas entre instruções do tipo R e do tipo I.

Opcode	Mnemônico	Tipo	Operação
0000	ADD	R	$rd = rs1 + rs2$
0001	SUB	R	$rd = rs1 - rs2$
0010	AND	R	$rd = rs1 \& rs2$
0011	OR	R	$rd = rs1 rs2$
0100	XOR	R	$rd = rs1 \wedge rs2$
0101	NOT	R	$rd = \sim rs1$
0110	SHL	R	$rd = rs1 \ll 1$
0111	SHR	R	$rd = rs1 \gg 1$
1000	ADDI	I	$rd = rs1 + \text{sign_extend}(imm)$
1001	LOAD	I	$rd = \text{mem}[rs1 + \text{sign_extend}(imm)]$
1010	STORE	I	$\text{mem}[rs1 + \text{sign_extend}(imm)] = rd$
1011	BEQ	I	if $(rd == rs1)$ $PC = PC + \text{sign_extend}(imm)$
1100	JUMP	I	$PC = \text{sign_extend}(imm)$

5.1 Formato das instruções

Todas as instruções possuem 32 bits. No tipo R, os campos são opcode [31:28], rd [27:24], rs1 [23:20], rs2 [19:16] e bits livres [15:0]. No tipo I, os campos são opcode [31:28], rd [27:24], rs1 [23:20] e imediato [19:0].

O imediato de 20 bits é estendido com sinal para 32 bits antes de ser usado pela ULA ou pela lógica de cálculo do PC, permitindo immediatos negativos e desvios retroativos.

5.2 Convenções de registradores

Registrador	Convenção
R0	Zero hardwired, sempre 0
R1 a R13	Uso geral
R14 (RA)	Endereço de retorno
R15 (RB)	Base pointer / stack pointer

6. Descrição dos Módulos Implementados

6.1 ULA (ula.h)

A ULA é um módulo combinacional que recebe dois operandos de 32 bits (A e B) e um código de operação de 4 bits (ctrl). Ela produz o resultado de 32 bits e as flags zero, carry e negativo. Neste projeto, a flag zero é utilizada pela lógica de controle de fluxo da instrução BEQ; carry e negativo ficam disponíveis para observação e extensões futuras.

Código	Operação	Descrição
0x0	ADD	Soma
0x1	SUB	Subtração
0x2	AND	E lógico bit a bit
0x3	OR	OU lógico bit a bit
0x4	XOR	OU exclusivo bit a bit
0x5	NOT	Negação bit a bit de A
0x6	SHL	Deslocamento lógico à esquerda de A em 1 bit
0x7	SHR	Deslocamento lógico à direita de A em 1 bit

6.2 Banco de Registradores (banco_reg.h)

O banco de registradores possui 16 registradores de 32 bits, três portas de leitura e uma porta de escrita. O registrador R0 é fixo em zero, de modo que tentativas de escrita nele são ignoradas. A escrita é síncrona na borda de subida do clock, enquanto a leitura é combinacional.

Porta	Uso
rs1_addr / rs1_data	Primeiro operando padrão
rs2_addr / rs2_data	Segundo operando padrão
r3_addr / r3_data	Leitura de rd para STORE e BEQ

O terceiro porto é necessário porque, em STORE, o dado gravado na memória é o conteúdo de rd; em BEQ, a comparação correta é entre rd e rs1. Os registradores internos são modelados como `sc_signal<sc_uint<32>>`, tornando o processo de leitura sensível a alterações no conteúdo dos registradores. Essa correção resolve o problema de saídas desatualizadas no modelo SystemC, mas não implementa forwarding arquitetural.

6.3 Memória Genérica (memoria.h)

O módulo de memória possui 256 palavras de 32 bits, com endereçamento por palavra. A leitura é combinacional e a escrita é síncrona. O mesmo módulo é instanciado como memória de instruções e memória de dados. Quando leitura está desabilitada, a saída dado_saida é explicitamente zerada, evitando comportamento de latch.

6.4 Unidade de Controle (controle.h)

A unidade de controle decodifica a instrução de 32 bits, extraindo opcode, rd, rs1, rs2 e imediato. Com base no opcode, ela gera os sinais que controlam o datapath.

Sinal	Função
reg_write	Habilita escrita no banco de registradores
mem_read	Habilita leitura da memória de dados
mem_write	Habilita escrita na memória de dados
mem_to_reg	Seleciona dado da memória para write-back
use_imm	Seleciona imediato como operando B da ULA
branch	Indica instrução BEQ
jump	Indica instrução JUMP
ula_op	Código de operação enviado à ULA

As instruções tipo R habilitam reg_write e usam o próprio opcode como ula_op. ADDI habilita reg_write e use_imm. LOAD habilita reg_write, mem_read, mem_to_reg e use_imm. STORE habilita mem_write e use_imm. BEQ habilita branch e usa SUB na ULA. JUMP habilita jump.

6.5 Datapath (datapath.h)

O Datapath é o módulo de topo. Ele integra unidade de controle, banco de registradores, ULA, memória de instruções, memória de dados, PC, registrador IF/EX, multiplexadores, lógica de atualização do PC e flush.

Condição	Próximo PC
jump = true	PC = sign_extend(imm)
branch = true e zero = true	PC = PC + sign_extend(imm)
caso contrário	PC = PC + 1

MUX	Função
mux_ula_a	Seleciona rs1_data ou r3_data para o operando A da ULA
mux_ula_b	Seleciona rs2_data, imediato estendido ou rs1_data para o operando B
mux_store_data	Seleciona o dado gravado na memória em STORE
mux_wb	Seleciona entre resultado da ULA e dado da memória para write-back

Quando ocorre JUMP ou BEQ tomado, a instrução já buscada no estágio IF é descartada por flush. Isso é feito escrevendo zero no registrador IF/EX, inserindo um NOP no pipeline.

7. Correções e Melhorias Aplicadas

7.1 Controle da memória de instruções

A memória de instruções deve ser lida continuamente e nunca deve ser escrita durante a execução normal. A correção conectou a leitura a um sinal fixo true e a escrita a um sinal fixo false, evitando dependência indevida do reset ou corrupção por sinais como a flag zero da ULA.

7.2 Correção do STORE

A instrução STORE executa $\text{mem}[\text{rs1} + \text{sign_extend}(\text{imm})] = \text{rd}$. A correção adicionou o terceiro porto de leitura do banco de registradores para ler rd diretamente. Assim, o dado enviado à memória em STORE passou a vir de r3_data.

7.3 Reatividade do banco de registradores no SystemC

Na versão inicial, os registradores internos eram variáveis simples. Após uma escrita síncrona, o processo de leitura não era automaticamente reexecutado, podendo manter saídas antigas. A correção modelou os registradores como `sc_signal<sc_uint<32>>` e tornou a leitura sensível aos endereços e ao conteúdo de todos os registradores. Isso corrige a fidelidade do modelo SystemC, mas não equivale a implementar forwarding arquitetural.

7.4 Extensão de sinal do imediato

O imediato de 20 bits é convertido para valor com sinal e estendido para 32 bits antes do uso. Isso permite immediatos negativos em ADDI, offsets negativos em LOAD e STORE, desvios retroativos em BEQ e endereços absolutos com sinal em JUMP.

7.5 Correção do BEQ e flush

A instrução BEQ compara rd com rs1. Para isso, o operando A da ULA recebe r3_data, o operando B recebe rs1_data, e a ULA executa SUB. Se o resultado for zero, o branch é tomado. Quando o fluxo de controle muda por BEQ tomado ou JUMP, a instrução já buscada no estágio IF é descartada por flush.

8. Testbench e Validação

O arquivo testbench.cpp instancia o Datapath, carrega um programa na memória de instruções, aplica reset e executa 25 ciclos de clock. Durante a simulação, são impressos sinais internos como PC, instrução em EX, operandos da ULA, resultado da ULA e dado de write-back. Ao final, o testbench compara os valores dos registradores e da memória de dados com os resultados esperados.

O testbench cobre todas as instruções da ISA implementada e alguns casos de borda.

8.1 Programa de teste

End.	Instrução	Hex	O que testa
0	ADDI R1, R0, 5	0x81000005	R1 = 5
1	ADDI R2, R0, 3	0x82000003	R2 = 3

2	ADD R3, R1, R2	0x03120000	R3 = 8
3	SUB R4, R3, R1	0x14310000	R4 = 3
4	AND R5, R3, R2	0x25320000	R5 = 0
5	OR R6, R3, R2	0x36320000	R6 = 11
6	XOR R7, R3, R2	0x47320000	R7 = 11
7	NOT R8, R1	0x58100000	R8 = 0xFFFFFFFF
8	SHL R9, R2	0x69200000	R9 = 6
9	SHR R10, R3	0x7A300000	R10 = 4
10	ADDI R11, R0, -1	0x8B0FFFFFFF	R11 = 0xFFFFFFFF
11	ADD R0, R1, R2	0x00120000	tentativa de escrita em R0
12	STORE R3, 10(R0)	0xA300000A	mem[10] = 8
13	LOAD R12, 10(R0)	0x9C00000A	R12 = 8
14	ADD R0, R0, R0	0x00000000	NOP demonstrativo pós-LOAD
15	BEQ R3, R1, 1	0xB3100001	branch não tomado
16	BEQ R1, R1, 1	0xB1100001	branch tomado
17	ADDI R13, R0, 99	0x8D000063	deve ser descartada pelo flush
18	JUMP 18	0xC0000012	halt por loop infinito

8.2 Resultados esperados

Registrador/posição	Valor esperado	Observação
R0	0	hardwired zero
R1	5	resultado de ADDI
R2	3	resultado de ADDI
R3	8	R1 + R2
R4	3	R3 - R1
R5	0	8 & 3
R6	11	8 3
R7	11	8 ^ 3
R8	0xFFFFFFFF	~5
R9	6	3 << 1
R10	4	8 >> 1
R11	0xFFFFFFFF	imediato -1 com extensão de sinal
R12	8	valor carregado de mem[10]
R13	0	instrução descartada pelo flush
mem[10]	8	valor armazenado por STORE

8.3 Observação sobre o NOP após LOAD

O NOP inserido após o LOAD demonstra a convenção de programação necessária para evitar o load-use hazard: se uma instrução LOAD for seguida imediatamente por uma instrução que usa o registrador carregado, deve-se inserir um NOP entre elas. Neste

programa de teste específico, a instrução posterior relevante é BEQ R3, R1, 1, que não consome R12. Portanto, o NOP funciona mais como documentação da limitação do pipeline do que como correção de uma dependência real naquele ponto.

9. Limitações Conhecidas

9.1 Load-use hazard

O processador não possui forwarding nem unidade automática de stall. Por isso, uma instrução que use imediatamente o resultado de um LOAD pode ler o valor antigo do registrador. A solução adotada é inserir manualmente um NOP entre o LOAD e a primeira instrução que consome o valor carregado.

```
LOAD R1, 0(R0)
ADD R0, R0, R0 ; NOP
ADD R2, R1, R3 ; uso seguro de R1
```

9.2 Penalidade de branch tomado

Quando um BEQ é tomado, a instrução já buscada no estágio IF é descartada por flush. Isso gera uma bolha de 1 ciclo. Branches não tomados não introduzem essa penalidade, pois a instrução sequencial já buscada é a correta.

9.3 Ausência de forwarding

No nível de simulação SystemC, o uso de sc_signal no banco de registradores corrige a atualização das saídas após escrita síncrona. No nível arquitetural, porém, o processador não implementa bypassing de dados entre estágios. Assim, cargas seguidas de uso imediato ainda exigem NOP manual ou, em uma versão futura, uma unidade de stall/forwarding.

9.4 Capacidade limitada das memórias

Cada memória possui 256 palavras de 32 bits. Acessos fora desse intervalo retornam zero silenciosamente, sem gerar exceção ou sinal de erro.

10. Como Compilar e Executar

10.1 Dependências

Em Ubuntu/Debian, instale:

```
sudo apt update
sudo apt install libsystemc-dev gtkwave
```

10.2 Compilação e execução

```
make
./risco_sim
```

Também é possível compilar e executar em um único comando usando o alvo do Makefile:

```
make run
```

10.3 Visualização das formas de onda

A simulação gera o arquivo `risco_trace.vcd`. Para abri-lo no GTKWave, use:

```
gtkwave risco_trace.vcd
```

Ou use o alvo do Makefile:

```
make wave
```

10.4 Limpeza

```
make clean
```

11. Trabalhos Futuros

- Implementar forwarding/bypassing.
- Adicionar unidade de detecção de hazards e inserção automática de stalls.
- Testar branches com offset negativo.
- Expandir a ISA com BNE, BLT, BGE, multiplicação e divisão.
- Implementar chamadas de subrotina.
- Criar pipeline com mais estágios (IF, ID, EX, MEM, WB).
- Sinalizar exceções para endereços de memória inválidos.
- Melhorar a integração com GTKWave por meio de arquivos `.gtkw`.

12. Estrutura dos Arquivos

Arquivo	Descrição
<code>ula.h</code>	Unidade Lógico-Aritmética
<code>banco_reg.h</code>	Banco de registradores com 3 portas de leitura e 1 de escrita
<code>memoria.h</code>	Memória genérica de 256 palavras de 32 bits
<code>controle.h</code>	Unidade de controle e decodificação de instruções
<code>datapath.h</code>	Integração dos módulos, pipeline, PC, MUXes e flush
<code>testbench.cpp</code>	Programa de teste e verificação automática
<code>Makefile</code>	Script de compilação, execução, abertura do GTKWave e limpeza
<code>Datapath.jpeg</code>	Diagrama da arquitetura geral do datapath
<code>risco_trace.vcd</code>	Arquivo de formas de onda gerado pela simulação

13. Conclusão

O projeto apresentado implementa, em SystemC, uma arquitetura funcional simplificada de processador denominada RISCO. O processador possui ISA própria de 13 instruções, banco de 16 registradores de 32 bits, ULA com 8 operações, memórias separadas para instruções e dados, unidade de controle por decodificação direta e pipeline de dois estágios.

Ao longo do desenvolvimento, foram aplicadas diversas correções e melhorias: controle adequado da memória de instruções, correção do dado enviado ao STORE, reatividade do banco de registradores no modelo SystemC, extensão de sinal do imediato, roteamento correto do BEQ e flush do pipeline em desvios tomados ou JUMP.

O testbench cobre todas as instruções da ISA implementada e casos de borda como imediato negativo, escrita em R0 e flush causado por branch tomado. As limitações conhecidas, principalmente o load-use hazard e a penalidade de branch, são compatíveis com o caráter didático do pipeline de dois estágios sem forwarding. O projeto constitui uma base sólida para estudos de datapath, unidade de controle, pipeline, hazards, memória, registradores e simulação de hardware com SystemC.